

I / Préparation de la base de données : création, remplissage et contraintes d'intégrités

Dans cette première partie nous allons alors créer la base de données et vérifier le bon formatage des données et des cohérences de celle-ci.

1 / Création de la base de données

Pour créer la base de données, nous avons créé une base dans phpMyAdmin, puis nous avons importé les fichiers CSV de chaque table un à un.

Lors de l'ajout des tables, nous avons dû sélectionner le fichier CSV, préciser que les séparations étaient faites par des virgules (et non des points-virgules), et cocher la case « Mettre en en-tête la première ligne » pour obtenir le bon nom des colonnes.

2 / Vérification et correction des données

Pour les dates de fin de location manquantes, nous avons ajouté une fonction random pour avoir de la donnée :

```
UPDATE Location  
SET datef = DATE_ADD(dated, INTERVAL FLOOR(RAND() * 101) DAY);
```

Nous avons aussi vérifié le type des variables date pour faire en sorte que ce ne soit plus des VARCHAR :

```
ALTER TABLE location  
MODIFY dated DATE,  
MODIFY datef DATE;
```

Nous avons aussi vérifié que les valeurs de fin et de début de location soient cohérentes, que la date de début ne soit pas avant la date de fin :

```
UPDATE location  
SET datef = DATE_ADD(dated, INTERVAL FLOOR(RAND() * 30 + 1) DAY)  
WHERE datef = dated OR datef IS NULL;
```

Pour chaque table nous avons ajouté une primary KEY implémentée pour pouvoir utiliser facilement nos tables :

```
ALTER TABLE location ADD id INT AUTO_INCREMENT PRIMARY KEY FIRST;
```

Nous avons ensuite lié les tables client et location :

ALTER TABLE location ADD id INT AUTO_INCREMENT PRIMARY KEY FIRST;

The screenshot shows a SQL query execution interface. The query is: `SELECT email, COUNT(*) AS nb FROM proprietaire WHERE email IS NOT NULL AND email <> '' GROUP BY email HAVING nb > 1;`. Below the query, there are buttons for 'Profilage', 'Éditer en ligne', 'Éditer', 'Expliquer SQL', 'Créer le code source PHP', and 'Actualiser'. Below these buttons, there are input fields for 'Tout afficher', 'Nombre de lignes' (set to 25), 'Filtrer les lignes', and a search box 'Chercher dans cette table'. Below the search box, there is a button 'Options supplémentaires'. Below the button, there is a table with two columns: 'email' and 'nb'. The table contains two rows: 'juju@hotmail.com' with '2' and 'L67@gmail.com' with '2'.

email	nb
juju@hotmail.com	2
L67@gmail.com	2

Pour les doublons de mails, nous avons décidé de mettre à NULL toutes les valeurs en double pour rendre l'attribut mail du client unique, et éviter qu'un client reçoive un mail destiné à son homonyme de mail:

```
UPDATE proprietaire
SET email = NULL
WHERE codeP NOT IN (
  SELECT codeP_min FROM (
    SELECT MIN(codeP) AS codeP_min
    FROM proprietaire
    WHERE email IS NOT NULL AND email <> ''
    GROUP BY email
  ) AS uniques
)
AND email IN (
  SELECT email FROM (
    SELECT email
    FROM proprietaire
    WHERE email IS NOT NULL AND email <> ''
    GROUP BY email
    HAVING COUNT(*) > 1
  ) AS doublons
);
ALTER TABLE proprietaire ADD UNIQUE (email);
```

Nous avons ensuite ajouté une clé étrangère reliant les voitures à leur propriétaire :

```
ALTER TABLE voiture
ADD CONSTRAINT fk_voiture_proprietaire
FOREIGN KEY (codeP)
REFERENCES proprietaire(codeP);
```

Enfin, nous avons supprimé les numéros de permis, pour les doublons, afin d'éviter des fraudes (2 personnes ne peuvent pas avoir le même numéro de permis, c'est illégal)

3/ Dans cette partie nous avons ajoutés toutes les clés étrangères nécessaires pour relier les tables entres elles :

```
ALTER TABLE voiture
ADD CONSTRAINT fk_voiture_proprietaire
FOREIGN KEY (codeP) REFERENCES proprietaire(codeP)
ON UPDATE CASCADE ON DELETE SET NULL;
ALTER TABLE location
ADD CONSTRAINT fk_location_client
FOREIGN KEY (CodeC) REFERENCES client(CodeC)
ON UPDATE CASCADE ON DELETE SET NULL;
ALTER TABLE location
ADD CONSTRAINT fk_location_voiture
FOREIGN KEY (immat) REFERENCES voiture(Immat)
ON UPDATE CASCADE ON DELETE SET NULL;
```

Dans cette partie nous avons vérifié que les valeurs étaient cohérentes pour le prix et le nombre de places dans les voitures, compris entre 1 et 9 :

```
ALTER TABLE voiture
ADD CONSTRAINT chk_places CHECK (places BETWEEN 1 AND 9);
Le prix doit être positif :
ALTER TABLE voiture
ADD CONSTRAINT chk_prixJ CHECK (prixJ > 0);
```

Enfin, nous avons fini par vérifier les contraintes :

L'année de l'achat de la voiture doit être compris entre 1900 et 2025 :

```
ALTER TABLE voiture
ADD CONSTRAINT chk_achatA CHECK (achatA BETWEEN 1900 AND 2025);
```

Nous avons fini par mettre une contrainte sur l'email du propriétaire, pour forcer la présence du caractère '@' lors de l'ajout ou de la modification d'un mail.

```
ALTER TABLE proprietaire
ADD CONSTRAINT chk_email_format
CHECK (email LIKE '%@%');
```

4 / Vérification de la cohérence des données

TABLE_NAME	CONSTRAINT_NAME	CHECK_CLAUSE
voiture	chk_places	('places' between 1 and 9)
voiture	chk_prixJ	('prixJ' > 0)
voiture	chk_achatA	('achatA' between 1900 and 2025)

CONSTRAINT_NAME	CHECK_CLAUSE
chk_email_format	(`email` like _utf8mb4\'%@%\')

II/ Vues

Dans cette deuxième partie, nous allons traiter les différentes vues des clients pour la gestion d'affichage des informations.

1/ Création de la table V_client :

Pour cela nous avons utilisé la fonction left join afin d'inclure tous les clients qui n'ont pas de location, ensuite nous avons utilisé la fonction sum pour avoir la somme des différents kilométrages du client. Ensuite on passe toutes les valeurs NULL à 0 kilomètres.

```
SELECT
  c.CodeC,
  c.Prenom,
  c.Nom,
  c.age,
  IFNULL(SUM(l.km), 0) AS distance
FROM client c
LEFT JOIN location l ON c.CodeC = l.CodeC
GROUP BY c.CodeC, c.Prenom, c.Nom, c.age;
```

2/ Dans cette question nous allons tester de modifier les V_client afin de voir si cela est possible ou non :

UPDATE ... SET distance=...

Lorsque qu'on essaye de modifier V_client, l'attribut « client » est non modifiable et la colonne distance est calculée en fonction des locations.

```
Requête SQL : Copier

UPDATE V_Client
SET Nom = 'LESTIC'
WHERE CodeC = 'C254';

MySQL a répondu : 
#1288 - The target table V_Client of the UPDATE is not updatable
```

3/ Création d'une vue clients âgés de plus de 55 ans V_Client55 :

On cherche à afficher le prénom, le nom, le numéro de permis des clients ayant plus de 55 ans.

```
CREATE OR REPLACE VIEW V_Client55 AS
SELECT CodeC, Prenom, Nom, age
FROM client
WHERE age > 55;
```



	CodeC	Prenom	Nom	age
☐ Éditer ☐ Copier ☐ Supprimer	C657	Jean	Gabin	67
☐ Éditer ☐ Copier ☐ Supprimer	C659	Gerard	Delon	67
☐ Éditer ☐ Copier ☐ Supprimer	C667		Charlot	77
☐ Éditer ☐ Copier ☐ Supprimer	C671	Jean-Pierre	Premier	63
☐ Éditer ☐ Copier ☐ Supprimer	C672		Richard	57

4/ Dans cette question on insère un client en moins de 55 ans pour vérifier que le filtre de 55 ans fonctionne bien et que le client apparait uniquement dans la table V_client.

```
INSERT INTO V_Client55 (CodeC, Prenom, Nom, age)
VALUES ('C999', 'Louis', 'Martin', 50);
```

Le nouveau client ne s'affiche pas dans la table V_client55 car il a moins de 55 ans :



	CodeC	Prenom	Nom	age
☐ Éditer ☐ Copier ☐ Supprimer	C657	Jean	Gabin	67
☐ Éditer ☐ Copier ☐ Supprimer	C659	Gerard	Delon	67
☐ Éditer ☐ Copier ☐ Supprimer	C667		Charlot	77
☐ Éditer ☐ Copier ☐ Supprimer	C671	Jean-Pierre	Premier	63
☐ Éditer ☐ Copier ☐ Supprimer	C672		Richard	57

Cependant le client est bien ajouté à la table V_client avec tous les clients :

☐ Éditer ☐ Copier ☐ Supprimer	C999	Martin	Louis	50	NULL	NULL	NULL
--	------	--------	-------	----	------	------	------

III / Droits d'accès

Dans cette partie nous allons nous intéresser aux droits d'accès des différents utilisateurs à la base de données agence_de_voyage.

1) Création de la table ACCES avec un user_id auto incrémenté (Primary Key) et en fonction des différents accès, l'utilisateur peu plus ou moins modifier des données :

- Lecture (L)
- Ecriture (E)
- Update (U)
- Delete (D)
- Total (T)

Le champs login doit être unique pour être sûr qu'un nom d'utilisateur correspond à 1 mot de passe unique et donc à un compte unique .

```
CREATE TABLE ACCESS (  
  user_id INT AUTO_INCREMENT PRIMARY KEY,  
  login VARCHAR(30) NOT NULL UNIQUE,  
  password VARCHAR(100) NOT NULL,  
  access_level ENUM('L','E','U','D','T') NOT NULL  
);
```



The screenshot shows a database management interface. At the top, it says "DESCRIBE ACCESS;" and has links for "[Éditer en ligne]", "[Éditer]", and "[Créer le code source PHP]". Below these is a button labeled "Options supplémentaires". The main part of the screenshot is a table showing the structure of the ACCESS table.

Field	Type	Null	Key	Default	Extra
user_id	int	NO	PRI	NULL	auto_increment
login	varchar(30)	NO	UNI	NULL	
password	varchar(100)	NO		NULL	
access_level	enum('L','E','U','D','T')	NO		NULL	

Pour vérifier le bon fonctionnement de la table ACCES, nous avons ajouté différents user avec différent accès différents :

```
INSERT INTO ACCESS (login, password, access_level)  
VALUES  
( 'chloe', 'azerty', 'T'),  
( 'edouard', '1234', 'L'),  
( 'professeur', 'admin123', 'E')
```

user_id	login	password	access_level
1	chloe	azerty	T
2	edouard	1234	L
3	professeur	admin123	E

Création de la Primary key auto-incrémenté avec ajout de l'accès lecture uniquement par défaut :

```
CREATE TABLE ACCESS (
  user_id INT AUTO_INCREMENT PRIMARY KEY,
  login VARCHAR(50) NOT NULL UNIQUE,
  password VARCHAR(100) NOT NULL,
  access_level ENUM('L', 'E', 'U', 'D', 'T') DEFAULT 'L'
);
```

2/ Remplissage de la table ACCES pour donner accès au client avec un accès de lecture (L) par défaut. Les propriétaires quant à eux ont un accès écriture (E) sauf ceux n'ayant pas d'adresse email avec un mot de passe crée automatiquement avec la fonction MD5() qui permet de chiffrer une chaîne de caractère (ici le CodeC) en un entier hexadécimal de 32 caractères.

Pour les clients :

```
INSERT INTO ACCESS (login, password, access_level)
SELECT LOWER(CONCAT(Nom, ' ', Prenom)) AS login,
       MD5(CONCAT(Nom, Prenom)) AS password,
       'L' AS access_level
FROM client;
```

Pour les propriétaires :

```
INSERT INTO ACCESS (login, password, access_level)
SELECT email, MD5(email), 'E'
FROM proprietaire
WHERE email IS NOT NULL AND email <> '';
```

Pour préciser le cas des emails non référencés ou NULL, il faudrait que le responsable de l'agence de voyage contacte les propriétaires afin de leur demander de mettre à jour l'email.

20	martin.louis	950630d7ca7a0fad0a3d3f705e279eb1	L
21	test.edouard	1ebffde4e9228e553b3dbb040c94866f	L
23	bozo@gmail.com	889ac2d82063e2125250b7255cb3a684	E
24	Gi67@gmail.com	f6fc0bd5b5fb617cf226a04e4b8e82f0	E

On peut voir que la base de données est bien modifiée avec les accès des clients 'lecture' et si l'adresse email est correcte pour un propriétaire avec les droits d'accès 'écriture'. Le mot de passe est bien généré automatiquement avec la fonction MD5().

3/ Création de différents utilisateurs avec différents types d'accès.

A) Avec des droits de lectures limités sur quelques tables

```
CREATE USER 'chloe'@'localhost' IDENTIFIED BY 'chloe123';
GRANT SELECT ON agence_de_voyage.client TO 'lecteur'@'localhost';
GRANT SELECT ON agence_de_voyage.voiture TO 'lecteur'@'localhost';
```

Grants for chloe@localhost

```
GRANT USAGE ON *.* TO `chloe`@`localhost`
```

```
GRANT SELECT ON `agence_de_voyage`.`client` TO `chloe`@`localhost`;
```

```
GRANT SELECT ON `agence_de_voyage`.`voiture` TO `chloe`@`localhost`;
```

b) Avec des droits de modifications (insert et update) sur quelques tables.

```
CREATE USER 'edouard'@'localhost' IDENTIFIED BY 'edouard123';
GRANT SELECT, INSERT, UPDATE ON agence_de_voyage.client TO 'edouard'@'localhost';
GRANT SELECT, INSERT, UPDATE ON agence_de_voyage.location TO 'edouard'@'localhost';
```

Grants for edouard@localhost

```
GRANT USAGE ON *.* TO `edouard`@`localhost`
```

```
GRANT SELECT, INSERT, UPDATE ON `agence_de_voyage`.`client` TO `edouard`@`localhost`;
```

```
GRANT SELECT, INSERT, UPDATE ON `agence_de_voyage`.`location` TO `edouard`@`localhost`;
```

c) Avec la totalité des droits. Ce dernier aura le droit de partager ses droits.

```
CREATE USER 'admin'@'localhost' IDENTIFIED BY 'admin123';
GRANT ALL PRIVILEGES ON agence_de_voyage.* TO 'admin'@'localhost' WITH GRANT OPTION;
puis on applique les changements
FLUSH PRIVILEGES;
```

GRANT permet d'accorder à un utilisateur des dro

Grants for admin@localhost

```
GRANT USAGE ON *.* TO `admin`@`localhost`
```

```
GRANT ALL PRIVILEGES ON `agence_de_voyage`.* TO `a...
```

its sur la base de données. FLUSH recharge les droits depuis les tables système pour que les nouveaux privilèges soient immédiatement pris en compte.

4/ Se connecter avec chaque utilisateur et vérifier les opérations : SELECT, INSERT, UPDATE et DELETE.

a) Lorsque je me connecte avec Chloé qui est lecteur :

```
#1142 - La commande 'INSERT' est interdite à l'utilisateur: 'chloe'@'localhost' sur la table 'client'
```

```
#1142 - La commande 'UPDATE' est interdite à l'utilisateur: 'chloe'@'localhost' sur la table 'client'
```

Les fonctions INSERT et UPDATE sont alors bloquées car cet utilisateur n'a accès que en tant que lecteur.

✓ Affichage des lignes 0 - 19 (total de 20, traitement en 0,0002 seconde(s).)

```
SELECT * FROM client;
```

Par contre, la fonction Select qui permet de lire fonctionne bien.

b) Lorsque je me connecte avec Edouard qui est éditeur :

✓ 1 ligne insérée. (traitement en 0,0004 seconde(s).)

```
INSERT INTO client (CodeC, Nom, Prenom, age, Permis, Adresse) VALUES ('C998', 'Test', 'edouard', 35, '78', 'Elancourt');
```

[Éditer en ligne] [Éditer] [Créer le code source PHP]

✓ 1 ligne affectée. (traitement en 0,0002 seconde(s).)

```
UPDATE client SET age = 22 WHERE CodeC = 'C654';
```

L'utilisateur a donc bien la possibilité de modifier et d'ajouter de nouvelle donnée dans la base de données.

```
#1142 - La commande 'DELETE' est interdite à l'utilisateur: 'edouard'@'localhost' sur la table 'client'
```

Par contre, il ne possède pas de droit de supprimer de la donnée dans la base.

- c) Lorsque on se connecte avec l'utilisateur admin qui possède les pleins droits sur la base de données :

✓ 1 ligne insérée. (traitement en 0,0001 seconde(s).)

```
INSERT INTO client (CodeC, Nom, Prenom, age, Perais, Adresse) VALUES ('C997', 'Admin', 'Total', 40, '777777', 'Nice');
```

✓ 1 ligne affectée. (traitement en 0,0002 seconde(s).)

```
UPDATE client SET age = 45 WHERE CodeC = 'C997';
```

✓ 1 ligne supprimée. (traitement en 0,0003 seconde(s).)

```
DELETE FROM client WHERE CodeC = 'C997';
```

L'admin peut modifier, supprimer, et ajouter de la donnée à la base, il a donc les pleins droits.

IV/ Accès concurrents

Dans cette partie nous travaillons sur la gestion des accès concurrents : lorsque deux utilisateurs ayant les mêmes droits d'accès modifient une même donnée.

Nous avons utilisé l'Use case suivant pour traiter les questions :

Nous allons ici simuler une mise à jour de données perdues. C'est à dire qu'un utilisateur connecté sur la session 1 va modifier le compteur kilométrique d'une voiture, et qu'en même temps, un autre sur la session 2, va modifier le compteur kilométrique de la même voiture.

Résultat : le nombre devrait être écrasé

- 1) Problème de mise à jour perdu :

Code pour le premier utilisateur :

```
START TRANSACTION;
```

```
SELECT compteur FROM voiture WHERE Immat = '11FG62';
```

```
UPDATE voiture SET compteur = 100180 WHERE Immat = '11FG62';
```

Code pour le second utilisateur :

```
START TRANSACTION;
```

```
SELECT compteur FROM voiture WHERE Immat = '11FG62'
```

```
UPDATE voiture SET compteur = 10200 WHERE Immat = '11FG62';  
COMMIT;
```

Puis nous réalisons un commit avec le premier utilisateur. Alors, Une des 2 mises à jour a été écrasée, le résultat obtenu est 100180, la donnée du second utilisateur est donc perdue.

Pour gérer ce genre de problème nous avons utilisé la méthode FOR UPDATE

```
START TRANSACTION;  
SELECT compteur  
FROM voiture  
WHERE Immat = '11FG62'  
LIMIT 1 FOR UPDATE;
```

Nous avons dû ajouter le LIMIT 1 avant le FOR UPDATE car sinon phpmyadmin met une limite automatique à la fin et cela ne fonctionnait pas

Puis on a mis :

```
UPDATE voiture  
SET compteur = compteur + 100  
WHERE Immat = '11FG62';  
COMMIT;
```

Nous avons alors exécuté ce code pour le deuxième utilisateur :

```
START TRANSACTION;  
SELECT compteur FROM voiture WHERE Immat = '11FG62' LIMIT 1 FOR UPDATE;
```

Et ensuite :

```
UPDATE voiture  
SET compteur = compteur + 100  
WHERE Immat = '11FG62';  
COMMIT;
```

Et la quand on veut afficher on obtient :

compteur
100400

Le résultat attendu est alors le bon car $100200 + 100 + 100 = 100400$ (+100 par session ouverte)

2) Nous allons faire le même schéma pour une lecture sale : c'est à dire un cas où une transaction lit une donnée modifiée par une autre transaction qui n'a pas encore été validée.

Dans la session 1 :

```
START TRANSACTION;  
UPDATE voiture  
SET compteur = compteur + 200  
WHERE Immat = '11FG62';
```

On ne fait pas de commit pour que l'utilisateur de la session 2 ne puisse pas voir la modification.

Puis on affiche la donnée dans la session 2, le résultat correspond à la donnée mise à jour par l'utilisateur de la session 1 on obtient 100600, or nous n'avons pas encore fait de COMMIT sur la session 1, ce qui signifie que la lecture est sale.

user_id	login	password	access_level
1	chloe	azerty	T
2	edouard	1234	L
3	professeur	admin123	E

Pour gérer ce problème de lecture sale, nous avons modifié le statut de la table qui était en MyISAM en InnoDB, cela a permis de passer d'un niveau d'isolation faible, a un niveau plus élevé.

3) L'utilisation de For Update :

FOR ... UPDATE permet de verrouiller en écriture des lignes d'une table, cette commande ne s'utilise que dans une transaction active il faut donc utiliser START TRANSACTION avant. Les autres transactions doivent donc attendre qu'il libère le verrou avec "commit" ou "rollback".

Par exemple, dans notre question 1, nous avons géré un problème de mise à jour perdue grâce à cette fonction. En effet, avant son utilisation nous avions des données qui s'écrasaient entre elles, ensuite nous avons utilisé :

```
START TRANSACTION;  
SELECT compteur FROM voiture WHERE Immat = '11FG62' FOR UPDATE;  
UPDATE voiture SET compteur = compteur + 150 WHERE Immat = '11FG62';
```

COMMIT;

Ainsi, lors du FOR UPDATE, la donnée est bloquée aux autres sessions jusqu'à l'utilisation de « COMMIT ». Cette approche empêche deux employés de modifier en même temps le kilométrage d'une même voiture et garantit la cohérence des données.

V/ PL/SQL & Trigger

Tout les codes de procédures stockées et de triggers sont présents en annexe

1) Analyse des locations et attribution d'une note à la location :

Nous avons créé une procédure stockée « attribuer_notes() » (A VOIR EN ANNEXE) qui permet d'attribuer une note à chaque location en respectant le tableau ci-dessous :

Critère	Notes
Km > 1000 et nombre de jours > 50 jours	5
Km > 500 et nombre de jours > 20 jours	4
Km > 100 et nombre de jours > 7jours	3
Km <= 100 et nombre de jours > 3jours	2
Sinon	1

Puis nous l'avons exécutée :

CALL attribuer_notes();

numloc	km	duree	note
C-45	37	60	2
B-260	37	33	2
A-125	70	85	2
C-46	876	22	4
B-261	876	61	4
A-126	678	36	4

Voici le résultat obtenu, on voit bien qu'une nouvelle colonne a été ajoutée et qu'elle contient les valeurs en rapport avec le kilométrage et la durée.

2) Ajouter de la champs avis en fonction de la note :

Nous avons créé une nouvelle procédure indépendante « attribuer_avis() » (VOIR EN ANNEXE) qui ajoute une colonne "avis", nous avons généré un avis en fonction des notes selon le tableau suivant :

Note	Avis attribué
5	Très satisfait
4	Satisfait
3	Assez satisfait
2	Peu satisfait
1	Un satisfait
NULL	Non évalué

Nous avons tout d'abord bien vérifié que "avis" avait la taille nécessaire en exécutant :
`ALTER TABLE location MODIFY avis VARCHAR(30);`

Comme vous pouvez voir ci-dessous la table affiche alors bien la note et l'avis de toutes les réservations.

numloc	note	avis
C-45	2	Peu satisfait
B-260	2	Peu satisfait
A-125	2	Peu satisfait
C-46	4	Satisfait
B-261	4	Satisfait
A-126	4	Satisfait
C-34	2	Peu satisfait
C-47	3	Assez satisfait

3) Analyse de donnée par client :

Pour cette question, nous avons créé une procédure stockée « analyse_client('CodeC') » (VOIR EN ANNEXE) qui permet d'analyser les locations d'un client, et qui prend en paramètre le codeC.

Cette procédure stockée permet de :

- Calculer le nombre total de ours
- Nombre de véhicule différents
- La moyenne des notes qu'ils ont eues

Exemple d'utilisation avec le client C654 :

Code Client	Durée totale (jours)	Nb véhicules différents	Moyenne des notes
C654	178	2	2.00

On obtient bien la durée totale de l'ensemble des locations, le nombre de véhicules différents loués et la note moyenne.

CodeC	immat	annee	mois	numloc	km	duree	villed	villea	dated	datef	note	avis
C654	11FG62	2015	4	C-45	37	60	Paris	Neuilly	2000-10-10	2000-12-09	2	Peu satisfait
C654	11FG62	2015	6	B-260	37	33	Neuilly	Paris	2000-10-10	2000-11-12	2	Peu satisfait
C654	75AZ92	2015	4	A-125	70	85	Montreuil	Montreuil	2010-02-01	2010-04-27	2	Peu satisfait

On voit bien que la durée totale correspond bien à la donnée du client : $60+33+85=178$
 Les différentes locations de ce client ont toute la note de 2 ce qui correspond bien à la note moyenne affichée.

4) Gestion de l'état des véhicules :

On souhaite maintenant ajouter un état dynamique à la table voiture son état pour ça on se base sur le tableau suivant :

Disponible	Libre pour une nouvelle location
En location	Actuellement louée
En réparation	Temporairement indisponible
Indisponible	Autre cas particulier

On ajoute une colonne dans la table voiture :

```
ALTER TABLE voiture
ADD COLUMN etat ENUM('Disponible', 'En location', 'En réparation', 'Indisponible')
DEFAULT 'Disponible';
```

Immat	etat
11FG62	Disponible

Nous avons ensuite ajouté un trigger qui permet de vérifier que la voiture est disponible avant de la louer (VOIR EN ANNEXE) .

→ Si elle l'est, son état passe automatiquement à "En location".
 → Quand la date de fin (datef) est atteinte ou qu'une mise à jour est effectuée.
 On remet automatiquement la voiture à "Disponible".

Nous allons faire le test avec une voiture. Tout d'abord nous allons vérifier son état, dans le cas de la voiture immatriculée 11FG62, la voiture est disponible (voir figure ci-dessus).
 On ajoute ensuite une nouvelle location sur cette voiture :

```
INSERT INTO location (numloc, CodeC, immat, duree, km, dated, datef)
VALUES ('L9999', 'C654', '11FG62', 7, 850, '2025-11-01', '2025-11-08');
```

Immat	etat
11FG62	En location

L'état de la voiture est alors modifié grâce à la réservation faite son statut est alors « en location ».

Puis, on tente de terminer la location en mettant la datef à CURDATE (la date du jour), on obtient bien que la voiture passe à l'état disponible.

Immat	etat
11FG62	Disponible

5) Solution automatique pour gérer l'historique de changement d'état d'une voiture.

Nous allons créer automatiquement un historique de changement d'état.

Nous allons créer une nouvelle table appelée « historique_etat », elle permet de garder une traçabilité : enregistrer l'ancien, le nouvel état et la date du changement.

Puis, nous créons un trigger automatique (VOIR CODE DU TRIGGER EN ANNEXE)

sur la table voiture, qui permet de suivre les changements : il s'exécute chaque fois que la colonne « état » change et ajoute une ligne dans la table « historique_etat ».

Pour vérifier que cela fonctionne, nous allons mettre une voiture à l'état "en location" afin de vérifier une nouvelle ligne de la table historique est chargée :

id	immat	ancien_etat	nouvel_etat	date_changement	commentaire
1	11FG62	Disponible	En location	2025-11-07 10:34:00	Changement automatique depuis trigger à 2025-11-07...

Le fonctionnement de notre trigger est validé et enregistre les différents états de la voiture avec ces changements.

VI / Application & data visualisation

Dans cette dernière partie après avoir traité toute la donnée, l'avoir formatée, ajouter des procédures et des triggers nous allons créer des visuels et une application pour manipuler et visualiser la donnée correctement.

- 1) Création d'une application permettant de se connecter à la base de données et d'exécuter des opérations CRUD.

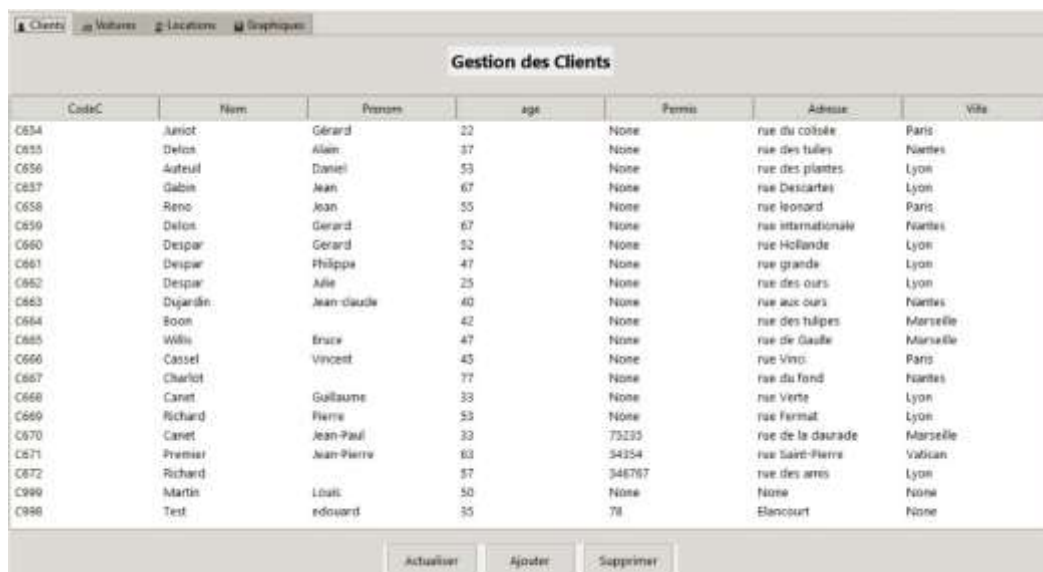
Cette application permet de faire les principales opérations CRUD (Create, Read, Update, Delete) sur les tables clients voitures et locations.

Nous avons utilisé le langage python et avons réalisé une interface graphique grâce à Tkinter.

Cette interface possède 3 onglets :

Clients :

- Affiche la liste complète des clients
- Permet d'ajouter un nouveau client
- Permet de supprimer un client existant
- Actualise les données affichées



CodeC	Nom	Prenom	age	Permis	Adresse	Ville
C634	Juniot	Gérard	22	None	rue du coléole	Paris
C635	Delos	Alain	37	None	rue des tulles	Nantes
C636	Auteul	Daniel	53	None	rue des plantes	Lyon
C637	Gabin	Jean	67	None	rue Descartes	Lyon
C638	Reno	Jean	55	None	rue leonard	Paris
C639	Delos	Gerard	67	None	rue internationale	Nantes
C660	Despar	Gerard	52	None	rue Hollande	Lyon
C661	Despar	Philippe	47	None	rue grande	Lyon
C662	Despar	Julie	25	None	rue des ours	Lyon
C663	Dujardin	Jean-claude	40	None	rue aux ours	Nantes
C664	Boon		42	None	rue des tulipes	Marseille
C665	Willis	Ernest	47	None	rue de Gaulle	Marseille
C666	Cassel	Vincent	45	None	rue Vimp	Paris
C667	Charlot		77	None	rue du fond	Nantes
C668	Canet	Guillaume	33	None	rue Verte	Lyon
C669	Richard	Pierre	53	None	rue Fermal	Lyon
C670	Canet	Jean-Paul	33	75235	rue de la daurada	Marseille
C671	Premier	Jean-Pierre	63	34354	rue Saint-Pierre	Vatican
C672	Richard		57	346767	rue des arnes	Lyon
C999	Martin	Loait	50	None	None	None
C998	Test	edouard	35	78	Blancourt	None

Voitures :

- Affiche la liste des véhicules

- Permet de supprimer une voiture ou d'actualiser l'affichage

Gestion des Voitures										
immat	modele	Marque	Categorie	couleur	places	achetA	compteur	prixl	codeP	etat
11FG62	Clio	Renault	base	Rouge	2	1998	10500	30.0	F19	En location
11RS75	Twingo	Renault	premium	Vert	4	2009	8565	30.0	F44	Disponible
11TR62	Clio	Renault	luxe	Rouge	2	1998	87230	30.0	F75	Disponible
11U62	Clio	Renault	luxe	Rouge	2	2005	4586	30.0	F25	Disponible
23AA46	Espace	Renault	familiale	Blanc	6	2014	8333	30.0	F19	Disponible
23TR62	404	Peugeot	premium	Rouge	2	1998	8720	30.0	F99	Disponible
32RS75	306	Peugeot	cabriolet	Vert	4	2010	4533	30.0	F61	Disponible
39YC59	C35	Citroen	utilitaire	Blanc	2	2002	67960	42.0	P01	Disponible
42RL75	T550	Ferrari	cabriolet	Vert	2	2013	46563	30.0	F89	Disponible
42RS75	megane	Renault	premium	Vert	4	2010	4583	30.0	F89	Disponible
56ER46	model 5	Teula	berline	Blanc	4	2015	854	30.0	F95	Disponible
69AZ92	205	Peugeot	berline	Blanc	2	2010	2987	30.0	F75	Disponible
718B37	C4	Citroen	familiale	Noir	4	2012	1258	30.0	F37	Disponible
71FO37	Caravan	Dodge	familiale	Blanc	6	2012	234258	30.0	F61	Disponible
71RT37	ES30	Ferrari	cabriolet	Rouge	2	2013	2359	30.0	F23	Disponible
75AZ92	205	Peugeot	cabriolet	Blanc	2	2010	17560	30.0	F75	Disponible
75BE37	Twingo	Renault	citadine	Noir	4	2011	9858	30.0	F99	Disponible
87AZ92	Zoe	Renault	citadine	Blanc	2	2010	37520	30.0	F75	Disponible
90AA46	508	Peugeot	familiale	Blanc	6	2000	7654	30.0	F14	Disponible

Actualiser
Supprimer

Locations :

- Liste les locations effectuées
- Permet de supprimer une ligne de location
- Affichage mis à jour en temps réel

Gestion des Locations										
numloc	CodeC	immat	km	duree	villed	villes	dated	datef	note	avis
1	C654	11FG62	2015	4	C-45	37	60	Paris	Neully	2000-10-10
2	C654	11FG62	2015	6	B-260	37	33	Neully	Paris	2000-10-10
3	C654	75AZ92	2015	4	A-125	70	85	Montreuil	Montreuil	2010-02-01
4	C653	11RS75	2015	4	C-46	876	22	Montreuil	Paris	2000-10-10
5	C655	11RS75	2015	6	B-261	876	61	Paris	Montreuil	2000-10-10
6	C655	42RL75	2015	4	A-126	678	36	Lyon	Lyon	2010-02-01
7	C655	718B37	2015	4	C-34	86	97	Neully	Neully	2010-02-01
8	C656	11TR62	2015	4	C-47	456	76	Neully	Montreuil	2000-10-10
9	C656	11TR62	2015	6	B-262	456	90	Montreuil	Neully	2000-10-10
10	C656	42RL75	2015	4	C-45	253	22	Lyon	Lyon	2010-02-01
11	C656	56AA46	2015	4	B-236	59	41	Paris	Paris	2010-02-01
12	C657	11TR62	2015	4	B-237	125	38	Montreuil	Montreuil	2010-02-01
13	C657	11U62	2015	4	C-48	23	67	Paris	Paris	2000-10-10
14	C657	11U62	2015	6	B-263	23	22	Paris	Paris	2000-10-10
15	C658	23AA46	2015	4	C-49	345	10	Montreuil	Montreuil	2000-10-10
16	C658	23AA46	2015	6	B-264	345	84	Montreuil	Montreuil	2000-10-10
17	C659	23TR62	2015	4	C-50	987	88	Neully	Neully	2000-10-10
18	C659	23TR62	2015	6	B-265	987	90	Neully	Neully	2000-10-10
19	C660	32RS75	2015	4	C-51	45	83	Paris	Paris	2000-10-10
20	C660	32RS75	2015	6	B-266	45	43	Paris	Paris	2000-10-10
21	C662	39YC59	2015	4	C-52	87	69	Montreuil	Paris	2000-10-10

Actualiser
Supprimer

Les différentes opérations sont réalisées grâce aux boutons en bas de page. Par exemple l'ajout d'un client:



Aj... — □ ×

CodeC

Nom

Prenom

age

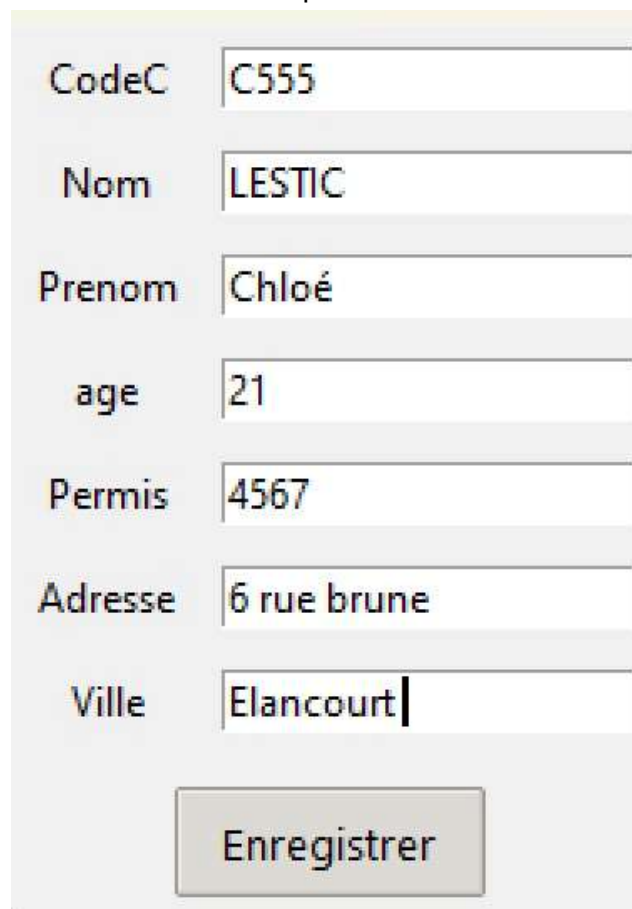
Permis

Adresse

Ville

Enregistrer

Voici la page qui s'ouvre quand nous cliquons sur "ajouter"
Lorsque qu'on remplit alors tous les champs :



CodeC

Nom

Prenom

age

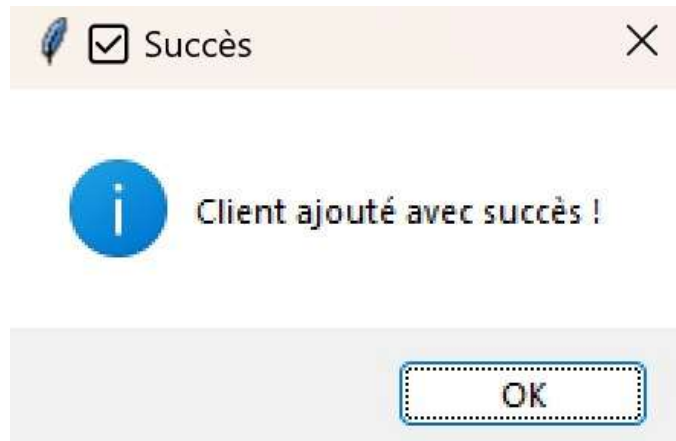
Permis

Adresse

Ville

Enregistrer

Un message pour mentionner le succès de l'opération s'affiche :

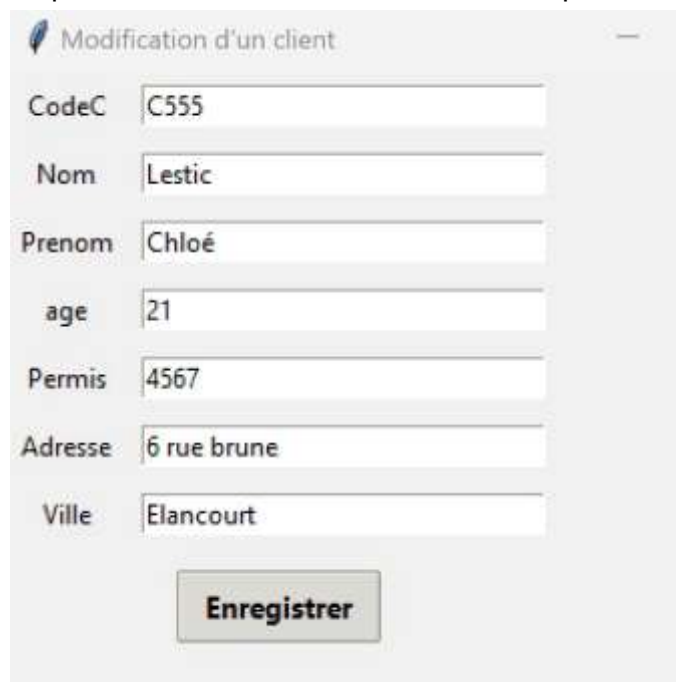


Lorsque qu'on vérifie l'ajout dans la base de données sur phpMyAdmin, on remarque que l'ajout du client a alors été fait :

C555	LESTIC	Chloé	21	4567	6 rue brune	Elancourt
------	--------	-------	----	------	-------------	-----------

Notre client C555 a été ajouté.

Ensuite nous avons la possibilité de modifier le client en cliquant sur « modifier »:

A form titled 'Modification d'un client' with a feather icon in the top left corner. The form contains several input fields: 'CodeC' with the value 'C555', 'Nom' with 'Lestic', 'Prenom' with 'Chloé', 'age' with '21', 'Permis' with '4567', 'Adresse' with '6 rue brune', and 'Ville' with 'Elancourt'. At the bottom center, there is a button labeled 'Enregistrer'.

Un formulaire pour modifier les informations est apparait pour que l'utilisateur puisse saisir la modification voulue.

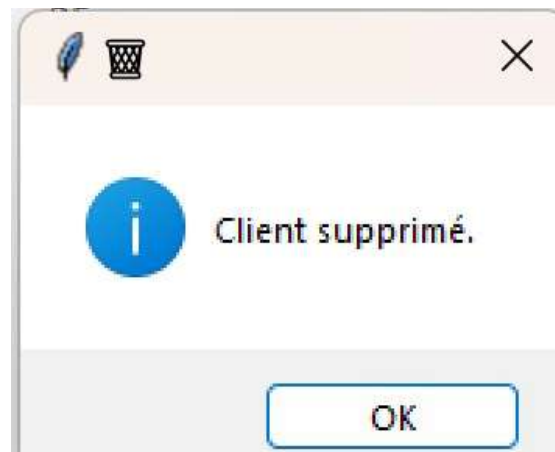


Lorsque la modification est faite une fenêtre s'affiche pour mentionner la modification. La base de données est alors modifiée comme on peut le voir ci-dessous l'âge passe alors à 22 ans :

C555	Lestic	Chloé	22	4567 6 rue brune	Elancourt
------	--------	-------	----	------------------	-----------

Finalement si on possède les codes de login de l'administrateur on peut alors supprimer les clients :

Il suffit de sélectionner un client parmi la liste et de cliquer ensuite sur le bouton supprimer. La fenêtre ci-dessous s'affichera et la base de données ne possèdera plus la ligne concernée ce client.



2) Création de visuels graphiques pour analyse de données.

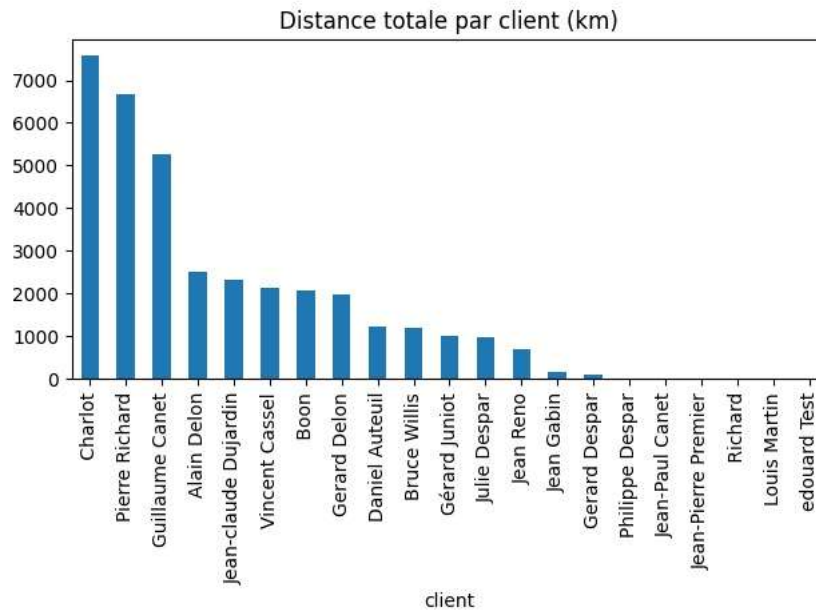
Dans cette partie, nous allons ajouter un onglet 'graphiques' dans notre application, affichant des visuels permettant faciliter l'analyse de notre agence de voyage.

1. Distance totale par client : Montre le total de kilomètres parcourus par chaque client

Question : Quels clients ont parcouru le plus de kilomètres au total lors de leurs locations ?

Requête pour obtenir le graphique :

```
SELECT c.Nom, SUM(l.km) AS distance
FROM client c
JOIN location l ON c.CodeC = l.CodeC
GROUP BY c.Nom;
```

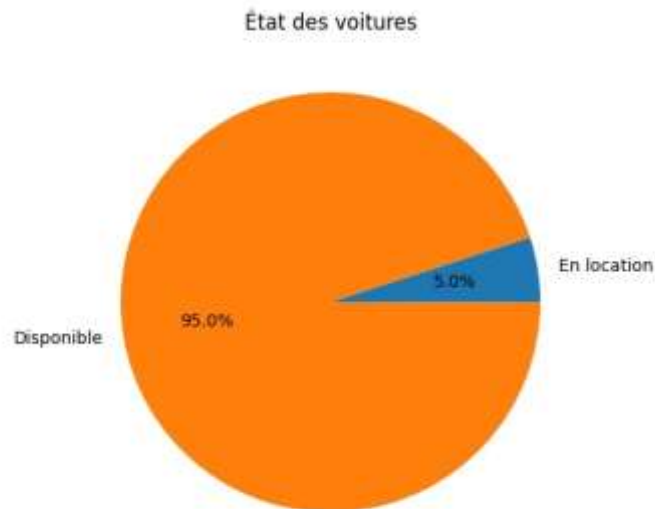


2. R partition des  tats des voitures : Indique la proportion de voitures disponibles, en location ou en r paration.

Question : Quelle est la r partition du parc automobile selon son  tat (disponible, en location, en r paration, etc.) ?

Requête pour obtenir le graphique :

```
SELECT  tat, COUNT(*) AS n
FROM voiture
GROUP BY  tat;
```

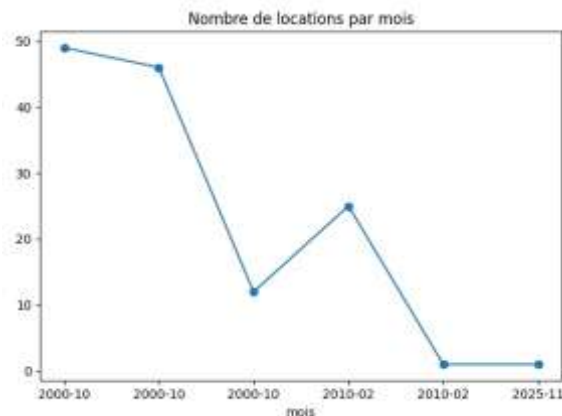


3. Nombre de locations par mois : Permet d'observer la saisonnalité de l'activité de location.

Question : Comment évolue le nombre de locations effectuées au fil des mois ?

Requête pour obtenir le graphique :

```
SELECT DATE_FORMAT(dated, '%Y-%m') AS mois, COUNT(*) AS n
FROM location
GROUP BY mois
ORDER BY mois;
```

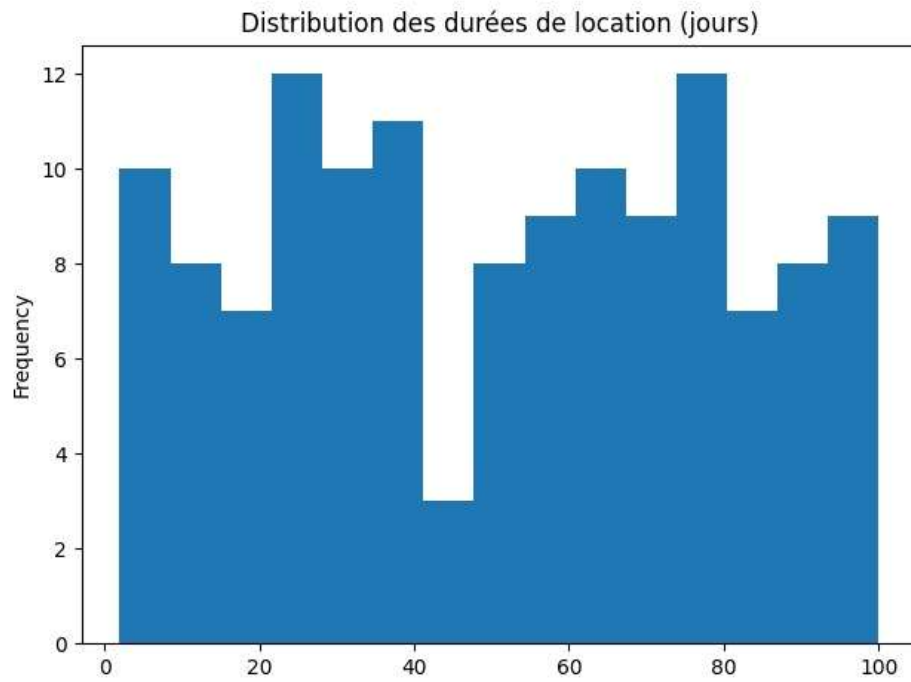


4. Distribution des durées de location : Montre la répartition des locations courtes et longues

Question : Quelles sont les durées de location les plus fréquentes parmi les clients ?

Requête pour obtenir le graphique :

```
SELECT duree
FROM location
WHERE duree > 0;
```



5. Moyenne des kilomètres par durée de location : Met en évidence la relation entre durée et distance parcourue.

Question : Quelle relation existe-t-il entre la durée de location et la distance moyenne parcourue ?

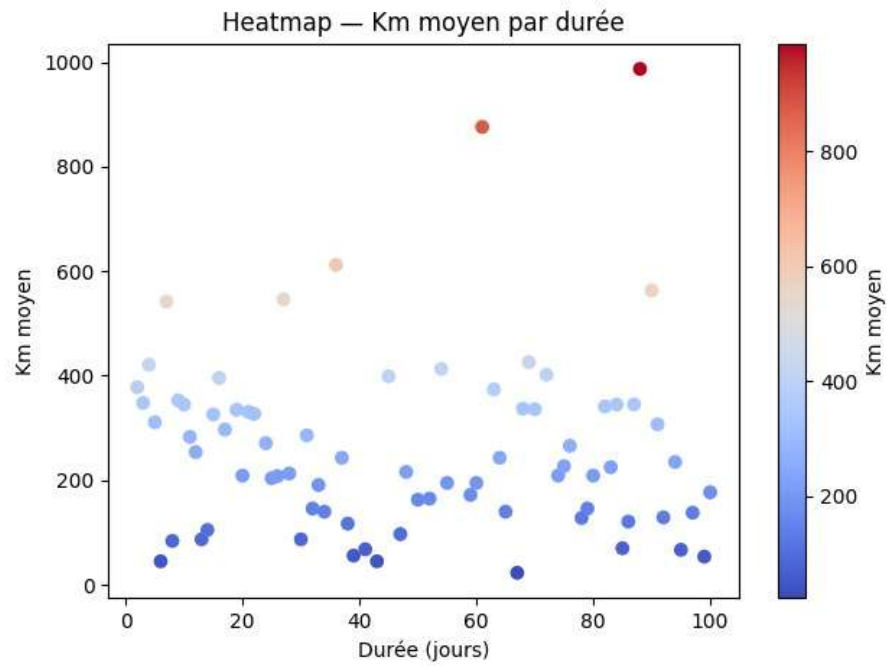
Requête pour obtenir le graphique :

```
SELECT duree, AVG(km) AS km_moy
```

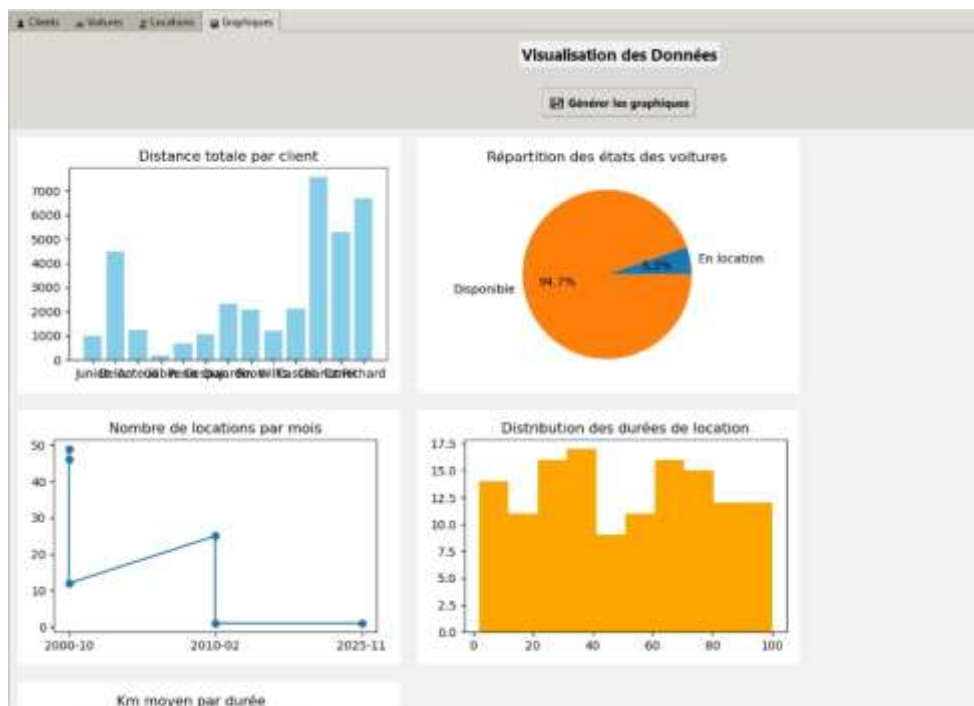
```
FROM location
```

```
WHERE duree > 0
```

```
GROUP BY duree;
```



L'onglet dans l'application évolue en fonction de la base de données et ressemble alors à ça :



Annexe :

Les procédures stockées / Triggers

Procédure stockée pour attribuer une note :

```
DELIMITER $$
CREATE DEFINER=admin@localhost PROCEDURE attribuer_notes()
BEGIN
    UPDATE location
    SET note =
    CASE
        WHEN km IS NULL OR duree IS NULL OR duree = 1 THEN NULL
        WHEN km > 1000 AND duree > 50 THEN 5
        WHEN km > 500 AND duree > 20 THEN 4
        WHEN km > 100 AND duree > 7 THEN 3
        WHEN km <= 100 AND duree > 3 THEN 2
        ELSE 1
    END;
END$$
DELIMITER ;
```

Procédure stockée pour attribuer un avis :

```
DELIMITER $$
CREATE DEFINER=admin@localhost PROCEDURE attribuer_avis()
BEGIN
    UPDATE location
    SET avis =
    CASE
        WHEN note >= 5 THEN 'Très satisfait'
        WHEN note = 4 THEN 'Satisfait'
        WHEN note = 3 THEN 'Assez satisfait'
        WHEN note = 2 THEN 'Peu satisfait'
        WHEN note = 1 THEN 'Insatisfait'
        WHEN note IS NULL THEN 'Non évalué'
        ELSE 'Non évalué'
    END;
END$$
```

DELIMITER ;

Procédure stockée pour analyser un client :

DELIMITER \$\$

CREATE DEFINER=admin@localhost PROCEDURE analyse_client(IN p_CodeC
VARCHAR(4))

BEGIN

DECLARE total_duree INT;

DECLARE nb_voitures INT;

DECLARE moyenne_note DECIMAL(4,2);

SELECT SUM(duree)

INTO total_duree

FROM location

WHERE CodeC = p_CodeC;

SELECT COUNT(DISTINCT immat)

INTO nb_voitures

FROM location

WHERE CodeC = p_CodeC;

SELECT AVG(note)

INTO moyenne_note

FROM location

WHERE CodeC = p_CodeC;

SELECT

p_CodeC AS 'Code Client',

total_duree AS 'Durée totale (jours)',

nb_voitures AS 'Nb véhicules différents',

ROUND(moyenne_note,2) AS 'Moyenne des notes';

END\$\$

DELIMITER ;

Trigger pour insérer une location :

DELIMITER //

CREATE TRIGGER trg_verif_voiture_avant_location

BEFORE INSERT ON location

```

FOR EACH ROW
BEGIN
    DECLARE v_etat VARCHAR(20);

    -- On récupère l'état actuel de la voiture
    SELECT etat INTO v_etat
    FROM voiture
    WHERE Immat = NEW.immat;

    -- Si la voiture n'est pas disponible, on bloque l'insertion
    IF v_etat <> 'Disponible' THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Erreur : cette voiture n'est pas disponible pour la location.';
    ELSE
        -- Sinon, on la passe en location
        UPDATE voiture
        SET etat = 'En location'
        WHERE Immat = NEW.immat;
    END IF;
END //

DELIMITER ;

```

Trigger après la fin d'une location :

```

DELIMITER //

CREATE TRIGGER trg_voiture_disponible_apres_location
AFTER UPDATE ON location
FOR EACH ROW
BEGIN
    -- Si la location est terminée (date de fin < aujourd'hui)
    IF NEW.datef <= CURDATE() THEN
        UPDATE voiture
        SET etat = 'Disponible'
        WHERE Immat = NEW.immat;
    END IF;
END //

DELIMITER ;

```

TRIGGER automatique pour l'historique :

DELIMITER //

```
CREATE TRIGGER trg_historique_etat_voiture
AFTER UPDATE ON voiture
FOR EACH ROW
BEGIN
    -- On vérifie que l'état a changé
    IF OLD.etat <> NEW.etat THEN
        INSERT INTO historique_etat (immat, ancien_etat, nouvel_etat, commentaire)
        VALUES (NEW.immat, OLD.etat, NEW.etat, CONCAT('Changement automatique
depuis trigger à ', NOW()));
    END IF;
END //
```

DELIMITER ;